ECE 122: Introduction to Programming for ECE – Spring 2026
Project 2: Student Course Manager

An Introduction to Object-Oriented Programming

Due Date: April 3rd, 2026

Upload Submissions to Gradescope

Objectives

This project is designed to introduce and assess the core learning objectives of Object-Oriented Programming (OOP). By completing this project, students will:

- Apply fundamental OOP principles (classes, objects, attributes, methods)
- Design and implement interacting classes (class definitions, constructors, instance variables)
- Use composition to create complex data structures (object containment, nested objects such as Course and CourseItems)
- Perform searching and filtering on collections (lists of objects, iteration, and conditional checks)
- Develop a modular, menu-driven program (class methods, user input handling, program flow control)

Project Description

The goal is to implement a text-based Student Course Manager. This tool allows a student to track courses (e.g., "ECE 122", "MATH 131") and specific items within those courses, like homework, quizzes, and exams. Each item tracks details such as title, category, due date, and score.

Project Files

main.py	Run this file to interact with the program. Implements the full menu-driven interface. You must complete all TODO items in this file.
course_management.py	Implement all required classes and methods described in the Class & Method Overview below.

Work Distribution

To ensure equal contribution, the following split must be followed:

Student 1	**Student 2**
CourseItem Class ● __init__() ● mark_complete() ● score_to_letter(percentage) **Course Class** ● set_weights(new_weights) ● display_weights() ● add_item(item) ● remove_item(title) ● calculate_grade() **CourseManager Class** ● __init__() ● add_course(course) **main.py** ● Menu options 1, 2, 3, 4, 5	**CourseItem Class** ● update_score(score) ● display_info() **Course Class** ● __init__() ● find_item(title) ● display_items() ● display_pending_items() **CourseManager Class** ● display_courses() ● find_course(name) ● find_course_by_code(code) **main.py** ● Menu options 6, 7, 8, 9, 10

Each student is responsible for implementing and testing the functions assigned to them. **Both students are expected to understand the full program structure and how all functions interact in main.py.**

course_management.py
CourseItem Class

__init__()	Initialize a new course item. **Attributes (must be included):** ● title – name of the item (e.g., "HW1", "Midterm") ● category – type of item (homework, quiz, exam, lecture notes, projects) ● due_date – due date as a string ● points_possible – max points for this item ● points_earned – numeric score (default: None until graded) ● completed – Boolean flag (default: False)
mark_complete()	Mark the item as completed. **Rules:** ● Sets the completed flag to True. ● This function must not print anything.
update_score(score)	Update the item's score. **Input:** ● score – numeric value to assign to points_earned ● Should accept any integer or float value **Rules:** ● Update self.points_earned with the score ● This function must not print anything.
display_info()	Return a formatted string describing this item. **Output format:** *category: title | Due: due_date | Score: score_text | Status: status* **Rules:** ● score_text is "Not graded" if points_earned is None, otherwise "**points_earned/points_possible**" ● status is "Completed" if completed is True, otherwise "Incomplete"

score_to_letter (percentage)	Convert a numeric percentage to a letter grade. **Input:** ● percentage **Output:** ● Returns a string corresponding to the percentage **Rules:** ● A >= 93 A- >= 90 B+ >= 87 B >= 83 B- >= 80 C+ >= 77 C >= 73 C- >= 70 D+>= 67 D >= 63 D- >= 60 F < 60

Course Class

__init__(name)	Initialize a course with a name and an empty list of items. **Input:** ● course_name – string identifier (e.g., "ECE 122") ● course_code – course code (e.g., "ECE122") ● instructor_name – name of the instructor **Rules:** ● Initialize self.items as an empty list ● self.weights should be a copy of DEFAULT_WEIGHTS
set_weights(self, new_weights)	Replace the course's category weights with new_weights **Inputs:** ● new_weights **Output:** ● Return True is valid, False otherwise **Rules:** ● The sum of all values must equal 100.0
display_weights (self)	Return a list of formatted strings showing current category weights. **Output:** ● Returns a list of strings **Format:** ● list[str]: e.g. ["Homework: 20.0%", "Quiz: 20.0%", ...]
add_item(item)	Add a CourseItem to this course. **Input:** ● A course item **Rules:** ● Appends the item to the internal items list. ● This function must not print anything.
find_item(title)	Search for a CourseItem by title. **Input:** ● item_title **Output:** ● Returns the matching CourseItem object, or None if not found. **Rules:** ● Search must be case-insensitive. ● This function must not print anything.

remove_item(title)	Remove a CourseItem by title. **Input:** ● item_title **Output:** ● Returns True if removed, False if not found. **Rules:** ● Search must be case-insensitive. ● This function must not print anything.
display_items()	Return a list of formatted strings for all items in this course. **Output:** ● Returns a list of items (str) **Rules:** ● Calls display_info() on each item. ● If no items exist, return ["No items found."].
display_pending_items()	Return a list of formatted strings for all incomplete items. **Output:** ● Returns a list of items (str) that haven't been completed yet. **Rules:** ● Filters items where completed == False. ● If all items are complete, return ["No pending items."].
calculate_grade()	Calculate and return the overall grade percentage for graded items. **Output:** ● **total** (CourseItem.**points_earned** / CourseItem.**points_possible**) *** 100** ● Returns 0.0 if no items have been graded or the total possible is 0. ● Otherwise return percentage_float and letter_grade_str **Rules:** ● Only items with non-None points_earned are included. ● This function must not print anything.

CourseManager Class

__init__()	Initialize the manager with an empty list of courses. **Rules:** ● Initialize self.courses as an empty list.
add_course(course)	Add a Course object to the manager's list. **Input:** ● Class name, class code, and professor name **Output** ● No output **Rules:** ● Accepts a Course object (not a name string). ● This function must not print anything.

display_courses()	Return a list of formatted strings for all courses.
	Output
	● Returns a list of all the courses
	Format:
	course_code: course_name (instructor_name)
	Rules:
	● If no courses exist, return ["No courses available."].
	`ECE 122: Intro. to Programming (Nikhil Saxena)`
	`Physics 152: Physics (David Hamilton)`
	`ECE 124: Intro. to Digital & Computer Systems (Maciej Ciesielski)`

find_course(name)	Search for and return a Course by name. **Input:** ● Course name **Output:** ● Returns the Course object, or None if not found. **Rules:** ● Search must be case-insensitive. ● This function must not print anything.
find_course_by_code(self, course_code)	Find and return a Course by course code. **Inputs:** ● course_code **Outputs:** ● Returns the Course object, or None if not found. **Rules:** ● Search must be case-insensitive. ● This function must not print anything.

main.py – Menu Overview

The following table describes each menu option that must be implemented in main.py. All input/output must match exactly.

	Option	**Behavior**
	Prompt User Code	Show all current courses, then prompt the user to enter a course code. ● ● Returns the matching Course object, or None if not found.

1	Add a new course	Prompt for course name, course code, and instructor name.
		● Create a Course object and add it via manager.add_course().
		● Print "Course added successfully."
		`Enter your choice: 1`
		`Enter course name: Intro. to Programming`
		`Enter course code: ECE 122`
		`Enter instructor name: Nikhil Saxena`
		`Course added successfully.`

2	View all courses	Call manager.display_courses() and print each returned string.
		`Enter your choice: 2`
		`ECE 122: Intro. to Programming (Nikhil Saxena)`
		`Physics 152: Physics (David Hamilton)`
		`ECE 124: Intro. to Digital & Computer Systems (Maciej Ciesielski)`

3	Add an item to a course	Prompt for course name, then find it with manager.find_course().
		● If not found, print "Course not found."
		● Otherwise, prompt for item title, category, due date, and points possible.
		● Create a CourseItem and add it via course.add_item().
		● Print "Item added successfully."
		`Enter your choice: 3`
		
		`Current courses:`
		`  ECE 122: Intro. to Programming (Nikhil Saxena)`
		`  Physics 152: Physics (David Hamilton)`
		`  ECE 124: Intro. to Digital & Computer Systems (Maciej Ciesielski)`
		`Enter course code: Math 132`
		`Course not found.`

		``
		Current courses:
		  ECE 122: Intro. to Programming (Nikhil Saxena)
		  Physics 152: Physics (David Hamilton)
		  ECE 124: Intro. to Digital & Computer Systems (Maciej Ciesielski)
		Enter course code: ECE 122
		Enter item title: Project 1
		Enter category (Homework/Quiz/Exam/Lecture Note/Project): Project
		Enter due date: 3/13/2026
		Enter points possible: 100
		Item added successfully.
		``

4	View all items in a course	Prompt for course name, then find it with manager.find_course().
		● If not found, print "Course not found."
		● Otherwise, call course.display_items() and print each string.
		``
		Enter your choice: 4
		
		Current courses:
		  ECE 122: Intro. to Programming (Nikhil Saxena)
		  Physics 152: Physics (David Hamilton)
		  ECE 124: Intro. to Digital & Computer Systems (Maciej Ciesielski)
		Enter course code: ECE 122
		Project: Project 1 | Due: 3/13/2026 | Score: Not graded | Status: Incomplete
		``

5	Mark an item as completed	Prompt for the course name and find it. If not found, print "Course not found." ● Prompt for item title and find it via course.find_item(). ● If not found, print "Item not found." ● Otherwise, call item.mark_complete() and print "Item marked as completed."

		<pre>Enter your choice: 5
		
		Current courses:
		  ECE 122: Intro. to Programming (Nikhil Saxena)
		  Physics 152: Physics (David Hamilton)
		  ECE 124: Intro. to Digital & Computer Systems (Maciej Ciesielski)
		Enter course code: ECE 122
		Enter item title: HW1
		Item not found.
		
		Course Management System
		1. Add a new course
		2. View all courses
		3. Add an item to a course
		4. View all items in a course
		5. Mark an item as completed
		6. Update an item's score
		7. View pending items
		8. Calculate course grade
		9. Customize category weights
		10. Exit
		Enter your choice: 5
		
		Current courses:
		  ECE 122: Intro. to Programming (Nikhil Saxena)
		  Physics 152: Physics (David Hamilton)
		  ECE 124: Intro. to Digital & Computer Systems (Maciej Ciesielski)
		Enter course code: ECE 122
		Enter item title: Project 1
		Item marked as completed.</pre>

6	Update an item's score	Prompt for the course name and find it. If not found, print "Course not found."
		● Prompt for item title and find it. If not found, print "Item not found."
		● Prompt for the numeric score and call item.update_score().
		● Print "Score updated successfully."
		<pre>Enter your choice: 6
		
		Current courses:
		  ECE 122: Intro. to Programming (Nikhil Saxena)
		  Physics 152: Physics (David Hamilton)
		  ECE 124: Intro. to Digital & Computer Systems (Maciej Ciesielski)
		Enter course code: ECE 122
		Enter item title: Project 1
		Enter score earned: 96
		Score updated successfully.</pre>

7	View pending items	Prompt for the course name and find it. If not found, print "Course not found."
		● Call course.display_pending_items() and print each string.
		`Enter your choice: 7`
		
		`Current courses:`
		`  ECE 122: Intro. to Programming (Nikhil Saxena)`
		`  Physics 152: Physics (David Hamilton)`
		`  ECE 124: Intro. to Digital & Computer Systems (Maciej Ciesielski)`
		`Enter course code: ECE 122`
		`No pending items.`

8	Calculate course grade	Prompt for the course name and find it. If not found, print "Course not found."
		● Call course.calculate_grade() and print "Course grade: X.XX%"
		● If no graded items, print "No graded items yet."
		`Enter your choice: 8`
		
		`Current courses:`
		`  ECE 122: Intro. to Programming (Nikhil Saxena)`
		`  Physics 152: Physics (David Hamilton)`
		`  ECE 124: Intro. to Digital & Computer Systems (Maciej Ciesielski)`
		`Enter course code: Physics 152`
		`No graded items yet.`
		
		`Enter your choice: 8`
		
		`Current courses:`
		`  ECE 122: Intro. to Programming (Nikhil Saxena)`
		`  Physics 152: Physics (David Hamilton)`
		`  ECE 124: Intro. to Digital & Computer Systems (Maciej Ciesielski)`
		`Enter course code: ECE 122`
		
		`Course Grade for ECE 122: Intro. to Programming`
		`  Weighted average : 96.00%`
		`  Letter grade  : A`
		
		`  Category breakdown:`
		`  Homework (20.0%): No graded items`
		`  Quiz (10.0%): No graded items`
		`  Exam (30.0%): No graded items`
		`  Lecture Note (5.0%): No graded items`
		`  Project (35.0%): 96.0/100.0 = 96.0%`

9	Customize category weights	Prompt for the course code and find it. If not found, print "Course not found."

		• Display the current category weights using course.display_weights()
		• Prompt the user to enter new weights for each category.
		• Verify that the sum of all category weights equals 100.
		• If valid, call course.set_weights(new_weights) and print "Weights updated successfully."
		• If invalid, print "Weights must sum to 100. No changes made."
		
		``
		Enter your choice: 9
		
		Current courses:
		  ECE 122: Python (Nikhil Saxena)
		Enter course code: ECE 122
		
		Current weights for ECE 122:
		  Homework: 20.0%
		  Quiz: 10.0%
		  Exam: 30.0%
		  Lecture Note: 5.0%
		  Project: 35.0%
		
		Enter new weights for each category (must sum to 100).
		Press Enter to keep the current value.
		  Homework (currently 20.0%): 15
		  Quiz (currently 10.0%):
		  Exam (currently 30.0%): 35
		  Lecture Note (currently 5.0%):
		  Project (currently 35.0%):
		Weights updated successfully.
		``

10	Exit	Print "Exiting program." and exit the loop.

		Course Management System 1. Add a new course 2. View all courses 3. Add an item to a course 4. View all items in a course 5. Mark an item as completed 6. Update an item's score 7. View pending items 8. Calculate course grade 9. Customize category weights 10. Exit Enter your choice: 10 Exiting program.

Grading Criteria (100 Points Total)

Test	Description	Points
Test 1 CourseItem Creation & Methods	**Functions tested:** __init__(), mark_complete(), update_score(), display_info() **What it checks:** ● CourseItem is created with correct attributes ● mark_complete() sets completed to True ● update_score() correctly updates points_earned ● display_info() output is correctly formatted	**15 pts**
Test 2 Course–Adding & Finding Items	**Functions tested:** add_item(), find_item(), display_items() **What it checks:** ● Items are added to the course correctly ● find_item() returns the correct item (case-insensitive) ● find_item() returns None when found ● display_items() calls display_info() on each item	**15 pts**
Test 3 Course–Removal & Pending Items	**Functions tested:** remove_item(), display_pending_items() **What it checks:** ● remove_item() removes the correct item and returns True ● remove_item() returns False when the item is not found ● display_pending_items() shows only incomplete items	**15 pts**
Test 4 Course–grade Calculation	**Functions tested:** calculate_grade() **What it checks:** ● Returns the correct percentage of graded items ● Ungraded items (points_earned=None) are excluded ● Returns 0.0 when no items have been graded	**15 pts**
Test 5 CourseManager – Adding & Finding Courses	**Functions tested:** add_course(), display_courses(), find_course() **What it checks:** ● Courses are added correctly ● display_courses() lists all courses ● find_course() returns the correct Course object (case-insensitive) ● find_course() returns None when found	**15 pts**
Criteria 1 Code Structure & Comments	Code must be properly commented, formatted, and follow OOP best practices.	**15 pts**

Criteria 2 Main Program Integration	All menu options in main.py must work correctly and handle errors such as "item not found" or invalid input.	**10 pts**

Submission Guidelines

- Submit course_management.py and main.py on Gradescope.
- Ensure all required classes and methods are correctly implemented.
- Only use programming concepts discussed in class up to OOP.
- Using AI or copying other students' code is not allowed and will result in grade deductions and disciplinary action.
- Code should be properly commented and formatted for readability.

This project is designed to be implemented incrementally. You are encouraged to run and test each class as you implement it. If you have any questions, reach out to the course TAs.

Happy Coding!!